



## Carátula para entrega de prácticas

# Laboratorio de computación salas A y B

*Profesor:* M.C Rene Adrian Davila Perez

---

*Asignatura:* Programación orientada a objetos

---

*Grupo:* 7

---

*No de* 3

*Práctica(s):*

---

*Integrante(s):* 323036285

323279866

323209432

---

323161891

323063609

314306908

---

*No. De Lista o* 3

*brigada*

---

*Semestre:* 2027-1

---

*Fecha de* 11/ 09 / 2026

*entrega:*

---

*Obervaciones:*

---

**CALIFICACIÓN:** \_\_\_\_\_

# Índice

|                  |   |
|------------------|---|
| 1. Introducción  | 2 |
| 2. Marco Teórico | 3 |
| 3. Desarrollo    | 4 |
| 4. Resultados    | 8 |
| 5. Conclusiones  | 8 |

# 1. Introducción

Actualmente los estudiantes de diferentes áreas usan los modelos de lenguaje a gran escala para realizar investigaciones, lo que genera un gran problema ya que muchos estudiantes usan la información sin verificarla y sin saber si es correcta o no, afectando la calidad y confiabilidad de trabajos académicos.

Al mismo tiempo, al iniciar en un nuevo lenguaje de programación al estudiantado se le presentan problemas que tienen múltiples soluciones, lo cuál se vuelve un problema a la hora de programar, ya que si no se manejan correctamente las estructuras más simples del lenguaje, un programa muy simple puede terminar siendo mucho más complejo de lo que se esperaba.

Resolver estos problemas nos ayudará a usar mejor a los LLM a la hora de realizar investigaciones, consiguiendo una mejor calidad y más veracidad en nuestros trabajos académicos, también nos ayudará a comprender mejor el lenguaje de programación Java, y practicar para encontrar las soluciones más simples y óptimas.

## Objetivos

- Verificar la información proporcionada por un LLM, para aumentar la calidad y veracidad en los trabajos académicos y discernir lo que es útil.
- Elaborar un programa donde se empleen diferentes métodos y estructuras, para lograr su entendimiento y emplearlos de la forma más sencilla posible.

## 2. Marco Teórico

### 1. Clases

Usamos únicamente el paquete `java.util` que contiene la clase `Scanner`, la cual utilizamos para capturar los datos ingresados por el usuario. En particular, empleamos el método `nextInt()` para leer el número de 5 dígitos. [4]

### 2. Estructuras de control

Se emplean dos estructuras de control:

- a) **Estructura while:** se utiliza para validar la entrada. Mientras la variable `num` no tenga 5 dígitos (`num < 10000 || num > 99999`), el ciclo vuelve a pedir el número. Esto funciona porque cualquier número de 5 dígitos debe ser mayor o igual a 10000 y menor o igual a 99999.
- b) **Estructura if-else:** se usa en dos ocasiones. La primera dentro del método `esPalindromo`, para retornar `true` o `false` según si cumple o no con la condición de ser palíndromo. La segunda en el `main`, para mostrar el mensaje correspondiente según el resultado de `esPalindromo`. [5]

### 3. División y módulo

En Java, cuando se dividen dos valores de tipo entero, el resultado también es un entero y no se redondea. Por ejemplo:

$$73254/10 = 7325$$

El operador módulo devuelve el residuo de una división. Por ejemplo:

$$73254 \% 10 = 4$$

Usando estas operaciones podemos obtener los dígitos de un número de 5 cifras:

- a) Primer dígito: `num / 10000`  
Al dividir entre 10000 se eliminan los últimos 4 dígitos y queda sólo el primero.
- b) Último dígito: `num % 10`  
El residuo de dividir entre 10 corresponde al último dígito.
- c) Segundo dígito: `(num / 1000) % 10`  
Primero se divide entre 1000 para quitar los últimos 3 dígitos y luego se aplica módulo 10 para obtener el segundo dígito.
- d) Cuarto dígito: `(num / 10) % 10`  
Primero se divide entre 10 para quitar el último dígito y luego se aplica módulo 10 para obtener el cuarto dígito.

### 4. Método `public static boolean esPalindromo(int num)`

Es un método que contiene la validación del palíndromo y tiene las siguientes características:

- `public`: significa que puede ser invocado desde cualquier parte. [3]
- `boolean`: significa que devuelve únicamente `true` o `false`. [1]
- `(int num)`: recibe como parámetro el número de 5 dígitos.

Dentro de `esPalindromo` se compara el primer dígito con el último y el segundo con el cuarto usando división y módulo. Si ambas comparaciones son verdaderas retorna `true`, en caso contrario retorna `false`. El dígito central no se compara porque no afecta a la condición de palíndromo.

## 5. Clase String

En Java las cadenas de caracteres son objetos pertenecientes a la clase `String`. Esta clase permite usar una amplia variedad de métodos para realizar operaciones de manipulación y tratamiento de cadenas de caracteres habituales en un programa. [2]

# 3. Desarrollo

Para éste punto de la práctica se le preguntó a un LLM acerca del funcionamiento de un `hashmap` y `treeset` y se le pidieron 3 ejemplos comunes de aplicación, a continuación se muestra su respuesta

## 1. HashMap y TreeSet

Para éste punto de la práctica se le preguntó a un LLM acerca del funcionamiento de un `HashMap` y `TreeSet` y se le pidieron 3 ejemplos comunes de aplicación, a continuación se muestra su respuesta:

### Funcionamiento Teórico y Técnico

#### ▪ `HashMap<K, V>`(Familia Map)

Estructura y concepto: Pertenece a la categoría de mapas o arreglos asociativos (`<llave, valor>`). En esta estructura las llaves son únicas (no se permite duplicidad de claves), mientras que los valores asociados sí pueden repetirse.

Mecanismo interno: No mantiene ningún orden de inserción ni ordenamiento natural de sus elementos. Emplea internamente una función hash calculada a partir de los métodos heredados de la clase `Object` (`hashCode()` y `equals()`) para dispersar las claves en una tabla y permitir inserciones, eliminaciones y búsquedas de acceso directo en tiempo promedio constante  $O(1)$ .

Métodos principales: `put(clave, valor)`, `get(clave)`, `containsKey(clave)`.

#### ▪ `TreeSet<E>`(Familia Set)

Estructura y concepto: Pertenece a la categoría de conjuntos (`Set`), no admite duplicados.

Mecanismo interno: A diferencia de `HashSet`, mantiene sus elementos ordenados de forma ascendente o natural. Se implementa mediante un árbol

binario balanceado (Rojo-Negro). Las operaciones básicas tienen complejidad  $O(\log n)$ .

Métodos principales: `add()`, `remove()`, `contains()`, además de métodos de rango como `first()`, `last()`, `headSet()`, `tailSet()`.

#### Diferencias Estructurales Directas

| Criterio              | HashMap<K, V><br>PDF                             | TreeSet<E><br>PDF                                    |
|-----------------------|--------------------------------------------------|------------------------------------------------------|
| Tipo de Colección     | Mapa / Diccionario asociativo (Map) PDF          | Conjunto ordenado (Set) PDF                          |
| Almacenamiento        | Pares de clave-valor (<llave, valor>) PDF        | Elementos individuales (solo valores) PDF            |
| Elementos duplicados  | Claves únicas, valores duplicados permitidos PDF | No permite duplicados bajo ninguna circunstancia PDF |
| Orden de los datos    | Sin orden garantizado PDF                        | Siempre ordenados de forma ascendente/alfabética     |
| Estructura subyacente | Tabla Hash con dispersión de llaves              | Árbol binario balanceado (Red-Black Tree)            |

## 2. Ejemplos comunes de aplicación

### a) Directorio Escolar o Catálogo de Alumnos por Número de Cuenta (HashMap)

```
import java.util.HashMap;
public class RegistroAlumnos {
    public static void main(String[] args) {
        HashMap<Integer, String> expediente = new HashMap<>();
        expediente.put(31820491, "Ana Martinez");
        expediente.put(31940582, "Carlos Ruiz");
        expediente.put(31789214, "Beatriz Soto");
        int cuentaBuscar = 31940582;
        if (expediente.containsKey(cuentaBuscar)) {
            System.out.println("Alumno: " + expediente.get(cuentaBus
        }
    }
}
```

### b) Padrón Depurado y Alfabético de Participantes (TreeSet)

```

import java.util.TreeSet;
public class ConcursoProgramacion {
    public static void main(String[] args) {
        TreeSet<String> participantes = new TreeSet<>();
        participantes.add("Perez Juan");
        participantes.add("Alvarez Sofia");
        participantes.add("Torres Mario");
        participantes.add("Perez Juan"); // duplicado ignorado
        for (String participante : participantes) {
            System.out.println(participante);
        }
    }
}

```

c) Tabla de Puntuaciones y Clasificación de Torneo (TreeSet)

```

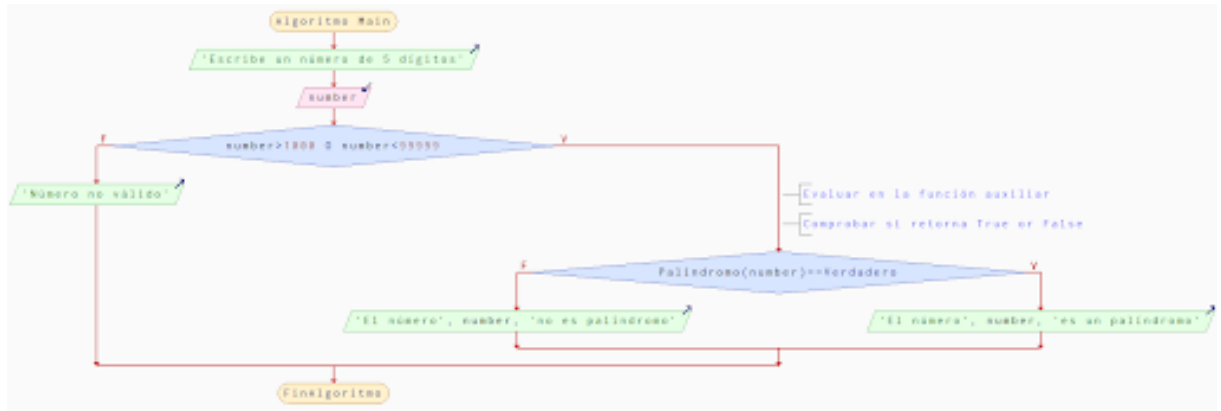
import java.util.TreeSet;
public class MarcadorJuego {
    public static void main(String[] args) {
        TreeSet<Integer> marcadores = new TreeSet<>();
        marcadores.add(850);
        marcadores.add(1200);
        marcadores.add(450);
        marcadores.add(1200); // duplicado descartado
        System.out.println("Menor puntaje: " + marcadores.first());
        System.out.println("Puntaje líder: " + marcadores.last());
    }
}

```

### 3. Ejercicio Palíndromo

Para el segundo ejercicio se trabajó con estructuras condicionales y métodos para realizar el análisis de un número y verificar si es palíndromo o no.

Se siguió el siguiente algoritmo:



El programa se centra en el método estático `main`, siendo este el contacto con la máquina virtual de Java. En su declaración se usa el parámetro `String[] args`, un arreglo de cadenas que permite recibir argumentos desde la línea de comandos. A pesar de que no se usan estos argumentos en el programa, es necesario declarar así el método, ya que permite que la máquina virtual reconozca éste método como punto de entrada y realizar la ejecución del programa.

Mediante la importación de clase `Scanner`, del paquete `java.util`, se puede capturar el número que el usuario ingrese, y mediante un ciclo `while`, se valida que el número tiene 5 dígitos, para garantizar que se pueda analizar correctamente en el programa.

Posteriormente, se usa un método para verificar si el número es un palíndromo usando operaciones aritméticas. En este método se regresa el tipo de dato `boolean`. Tras realizar el análisis el método regresa `true` si es un palíndromo y `false` si no lo es, para finalmente, empleando otro método, imprimir si el número es un palíndromo o no lo es.



## 4. Resultados

Al ejecutar el programa del palíndromo, se le pide al usuario ingresar un número entero positivo de 5 dígitos (a elección del usuario).

```
Programa para determinar si un número de 5 dígitos es un palidromo o no.  
Favor de ingresar un número entero positivo de 5 dígitos: 123456
```

Si el valor ingresado no corresponde a un entero positivo de 5 dígitos, el programa vuelve a solicitar el número hasta que el usuario ingrese una entrada válida.

```
Programa para determinar si un número de 5 dígitos es un palidromo o no.  
Favor de ingresar un número entero positivo de 5 dígitos: 123456  
Error: el número debe tener 5 dígitos.  
Ingresa el nuevo número:
```

Una vez ingresada una entrada válida, el programa evalúa si el número es un palíndromo o no y finaliza su ejecución.

```
Programa para determinar si un número de 5 dígitos es un palidromo o no.  
Favor de ingresar un número entero positivo de 5 dígitos: 123456  
Error: el número debe tener 5 dígitos.  
Ingresa el nuevo número: 12345  
12345 no es un palíndromo.
```

En las siguientes capturas se observan casos de prueba con números enteros positivos de 5 dígitos que corresponden a un palíndromo.

```
Programa para determinar si un número de 5 dígitos es un palidromo o no.  
Favor de ingresar un número entero positivo de 5 dígitos: 12321  
12321 es un palíndromo.
```

```
Programa para determinar si un número de 5 dígitos es un palidromo o no.  
Favor de ingresar un número entero positivo de 5 dígitos: 16061  
16061 es un palíndromo.
```

```
Programa para determinar si un número de 5 dígitos es un palidromo o no.  
Favor de ingresar un número entero positivo de 5 dígitos: 87078  
87078 es un palíndromo.
```

## 5. Conclusiones

El primer objetivo de esta práctica se cumplió ya que se logró comprender lo que es HashMap y TreeSet, y el alumnado pudo discernir si la información proporcionada era útil o no. El segundo objetivo también fue cumplido ya que se logró entender mejor como funcionan los métodos y estructuras, y se encontró una solución al problema sencilla y eficiente.

## Referencias

- [1] Portal Académico CCH-UNAM. *Métodos - Principios de Programación Orientada a Objetos*. <https://portalacademico.cch.unam.mx/cibernetica2/principios-programacion-orientada-a-objetos/metodos>. Accedido: 10 de septiembre de 2026.
- [2] Facultad de Ingeniería. *Manual de prácticas del Laboratorio de Programación Orientada a Objetos*. Documento en formato PDF. Emitido el 20 de enero de 2017. 2017.
- [3] Programarya. *Aprende métodos en Java, con ejemplos y ejercicios*. <https://www.programarya.com/Curso>. Accedido: 10 de septiembre de 2026.
- [4] Sarreplec. 7.3.- *Entrada por teclado. Clase Scanner. / PROG03. Contenidos Unidad 3*. [https://sarreplec.caib.es/pluginfile.php/10221/mod\\_resource/content/3/73\\_entrada\\_por\\_teclado\\_clase\\_scanner.html](https://sarreplec.caib.es/pluginfile.php/10221/mod_resource/content/3/73_entrada_por_teclado_clase_scanner.html). Accedido: 10 de septiembre de 2026.
- [5] Throphic. *Estructuras de control en Java*. [https://dat-science.com/estructurasde-control-en-java/#Estructura\\_if](https://dat-science.com/estructurasde-control-en-java/#Estructura_if). Accedido: 10 de septiembre de 2026.